Detailed and Updated Summary:

3D Reconstruction in Robotics

Computers and Graphics 130 (2025) 104256

NECKER
Based on the original PDF + some clarifications

November 17, 2025

Contents

Abstract

This document presents a detailed analysis of advancements in the field of 3D reconstruction in robotics, a crucial domain in contemporary research. The ability of robots to interact with and understand their environment is significantly enhanced by the integration of 3D reconstruction techniques, which draw inspiration from natural evolution and human perception. Cutting-edge methodologies such as SLAM, NeRF, Gaussian Splatting, 3D Semantic Reconstruction, Multi-View Reconstruction, and Amodal Reconstruction are explored, highlighting their synergistic contributions to robotic autonomy in complex and dynamic environments.

1 Technologies and Modeling Techniques

1.1 Pillar Technologies

State-of-the-art developments in this field include Neural Radiance Fields (NeRFs), which are a formidable method for capturing precise scene geometries and synthesizing novel views with remarkable detail. Alongside NeRFs, Simultaneous Localisation and Mapping (SLAM) provides real-time mapping and localization of the environment, significantly enhancing the autonomy of robotic systems. **Gaussian Splatting (GS)** is also gaining popularity, often used with NeRF models, due to its high speed.

1.2 3D Modeling Techniques

The document also discusses the utility of several 3D modeling techniques:

- **Surfels (Oriented Disks)**: They are excellent for fast rendering of complex and detailed surfaces and are advantageous in real-time rendering and point cloud processing.
- **Polygonal Modeling**: Uses vertices and edges to form polygons and is superior in applications requiring precise geometric manipulations, such as CAD and animation (more precise for rendering).

2 Related Work

The field of 3D reconstruction has been the subject of numerous studies and syntheses covering different techniques and applications.

2.1 Comprehensive Surveys on 3D Reconstruction

There are surveys that cover a wide range of methods, classifying them based on input modalities and effectiveness, and emphasizing their evolution over time.

2.2 Focused Reviews

More targeted reviews on specific frameworks or technologies are also available:

- In-depth analysis of the **SLAM** framework, detailing its main modules and the challenges faced by SLAM networks.
- Exploration of the challenges and applications of **3D LiDAR** technology within SLAM systems.

2.3 The Approach of the Present Work

This study distinguishes itself by focusing on the integration of advanced technologies like **NeRF** and **SLAM** within robotic systems to enhance perception and interactivity. The objective is to explore how these technologies synergistically improve robotic capabilities in dynamic environments.

3 Simultaneous Localisation and Mapping (SLAM)

3.1 Overview

SLAM is a crucial technique that allows devices to **create a 3D map** while **localizing** themselves within it and computing trajectories for autonomous navigation. This technique is essential in robotics for generating **real-time sparse 3D point cloud maps**, enabling robots to adapt to their surroundings. SLAM has a wide range of applications in underwater, planetary, and indoor environments. It can utilize different sensors, including RGB-D and LiDAR, and can process data using CPU storage or in the **cloud**.
The SLAM Process is Divided into 2 Phases in a Continuous Loop:

1. **Mapping Phase:** the map is created using various methods such as **NeRF** or **GS** detailed in this paper.
2. **Self-Localization Phase:** the robot localizes itself thanks to the built map and then restarts the loop by updating or expanding the map.

3.2 Indoor Reconstruction

The integration of SLAM techniques into indoor environments has led to significant advancements:

- **Hardware Efficiency**: An **FPGA**- and **CPU**-based system can generate SLAM reconstructions of indoor environments using LiDAR to create point cloud maps and TSDF Fusion for dense reconstructions. This approach offers high-quality reconstructions but requires expensive and customized hardware.
- **Edge Computing (cloud computing but with close and dedicated servers)**: Used for real-time scene reconstruction, processing data on a powerful *edge* device and fusing maps in the cloud. This method, however, struggles with scenes that are too *cluttered* (crowded).
- **Handling Dynamic Elements**: To eliminate problems caused by dynamic objects (moving independently of the observer) in indoor environments, a method that leverages geometric residuals can be used to reconstruct based only on static objects.

3.3 Underwater SLAM

SLAM is also extended to underwater environments using unmanned underwater vehicles (UUVs).

- **Multi-Sensor Integration**: Systems like **ORB-SLAM2** are combined with an acoustic odometer (DVL, gyroscope) to enable **sparse reconstructions** of submerged surfaces. A multi-sensor approach (DVL, gyroscopes, altimeters) improves navigation and mapping accuracy of AUVs in complex underwater environments.
- **Feature Enhancement**: **RU-SLAM** introduces **UWNet**, a specialized generator that improves key feature extraction in weak textures and degraded images, integrated into **ORB-SLAM3** to enhance robustness in complex underwater scenarios.
- **Autonomous Localization**: Depth mapping combined with semantic understanding improves autonomous localization when GPS is unavailable.

3.4 Resource-Constrained SLAM

In low-memory devices, a SLAM system is capable of performing **sparse reconstructions** on robots to produce **dense voxel-based maps** from sparse SLAM data, demonstrating efficiency for large-scale reconstructions despite some accuracy limitations.

- **Multi-Robot Systems**: A multi-robot system can distribute computations to *edge* devices and merge maps in the cloud, although this system may face challenges in map accuracy due to incomplete scans.
- **Sensor Integration**: The integration of LiDAR and RGB-D (**D** stands for depth) sensors allows dense 3D reconstruction using a small tracked robot, where LiDAR builds a 2D SLAM map and RGB-D data refines the reconstruction, though computational demands pose a challenge for real-time applications.

3.5 Collaborative SLAM Techniques

Collaborative SLAM uses multiple robots (for example, a drone and a terrestrial quadruped robot) for joint trajectory planning in chaotic environments.

- **Mapping Process**: The drone produces the **initial (sparse) SLAM point map**, which is subsequently voxelized to show the environment in relation to both robots, facilitating initial trajectory planning for the quadruped robot.
- **Map Update**: The terrestrial robot executes the initialized trajectory and updates the trajectory and the map with the onboard RGB-D camera if unseen objects are present in the scene.

3.6 Multi-Camera SLAM

Multi-camera SLAM systems offer several significant advantages over single-camera systems:

- **Improved Field of View**: Multiple cameras simultaneously capture different perspectives, mitigating the risk of losing track of *landmarks*, especially in dynamic or cluttered environments.
- **Better Depth Estimation**: Stereo or multi-camera configurations inherently obtain depth information through triangulation, leading to a **superior three-dimensional reconstruction** and **greater localization accuracy**.
- **Increased Robustness**: Multi-camera systems are more robust to occlusion and better suited to handle dynamic environments, unlike single-camera SLAM which can face challenges that can lead to significant localization errors.

For asynchronous multi-camera systems (images uncorrelated since the robot might have changed position, generally referring to sensors on the same robot), a system is required to align images to a **continuous time frame** by incorporating a continuous-time motion model. For synchronous systems, the concept of a **BundledFrame** (a virtual camera) allows the integration of measurements from all cameras, facilitating effective data fusion and accurate trajectory estimation (BundledSLAM).

3.7 SLAM and the Vision of NeRF and Splatting

Recent developments have integrated SLAM with cutting-edge rendering techniques to improve performance.

3.7.1 Integration with Gaussian Splatting (GS)

GS models a 3D scene as a continuous density field composed of **Gaussian primitives**.

- **Fast Rendering**: GS facilitates efficient rendering, allowing rapid map updates by adding Gaussians in previously unseen regions.
- **GS-SLAM**: Uses a *coarse-to-fine* strategy (from rough to fine) to select reliable 3D Gaussians, optimizing camera pose estimation and improving tracking stability and accuracy.
- **Active SLAM**: **AG-SLAM** is the first system to incorporate 3DGS representation into an **active SLAM** framework, enabling autonomous exploration and mapping of unknown environments in real time and with high quality.

3.7.2 Integration with NeRF

Traditional NeRF-based SLAM systems rely on a single Multi-Layer Perceptron (MLP) to represent the entire scene, which can lead to excessively *smooth* reconstructions and scalability challenges.

- **Scalability**: **NICE-SLAM** addresses these limitations by introducing a **hierarchical grid-based representation** for localized updates and better scalability.
- **Dynamic Robustness**: **RoDyn-SLAM** tackles dynamic scenes by integrating NeRF with the generation of *motion masks* to filter out invalid rays, blending optical flow masks and semantic masks to accurately distinguish between static and dynamic elements.

3.8 Quantitative SLAM Results

Real-time efficiency is a key metric in SLAM.

- **Time and Energy Efficiency**: **SLAM-Box** (LiDAR, FPGA + CPU) achieves an average scan time of **38 ms** and a power consumption of **13.8 W**.
- **Sensor Comparison (KITTI)**: Stereo camera systems (e.g., ORB-SLAM2 and RTABMap) achieve significantly lower TRMSE (Translation RMSE) values (around 2.81 and 2.817) compared to monocular approaches (e.g., ORB-SLAM3 with 16.878), highlighting the advantage of stereo data for trajectory estimation.
- **Accuracy (TUM RGB-D)**: RGB-D methods outperform monocular approaches. **NICE-SLAM** achieves an ATE RMSE of 2.6, significantly better than monocular methods.

4 Gaussian Splatting (GS) in Robotics

4.1 Overview

Gaussian Splatting (GS) is a technique used for 3D reconstruction, rendering, and data representation (see **Terminology** section).

4.2 Applications in Robotics

GS is applied to robotics for navigation and manipulation:

- **Safe Navigation**: **Splat-Plan** and **Splat-Loc** use GSplat maps for safe trajectory planning (through polytopal corridors: computes areas considered empty from the map) and robust pose localization (compares the map it has with what it sees to calculate the transformation and thus the pose, more robust than classic point clouds since it fills empty spaces). This approach allows global pose initialization without prior knowledge, leveraging the point cloud nature of GSplat scenes.
- **Semantic Manipulation**: *Object-centric* semantic integration allows initializing and collectively updating Gaussians with identical semantic labels. Gaussian splats integrate RGB details with geometric representation, improving semantic quality and facilitating real-time updates.
- **SLAM**: **Gaussian splat SLAM** can reconstruct tiny and even transparent objects, which are often difficult for traditional SLAM techniques, and achieves state-of-the-art results in novel view synthesis.
- **Human-Robot Interaction (HRI)**: GS is more effective than NeRF for enhancing HRI in high-risk contexts (such as disaster relief) due to its superior rendering speed.

4.3 Evolution of Gaussian Splatting

The evolution of GS techniques has led to improvements in efficiency and quality. Recent developments include:

- **View Consistency**: Implementing a *sorted splatting* mechanism to improve stability during rendering processes.
- **Surface Uniformity**: A *split* algorithm focused on improving surface uniformity and adherence, which facilitates more accurate point cloud extraction.
- **Detail Enhancement**: Integration of surface normals into the *rendering pipeline* to increase surface detail estimation (comparing them with normals calculated from sensors like LiDAR).
- **Anti-aliasing**: The introduction of a multi-scale approach (rendering in higher detail only as the robot approaches the object) to mitigate *aliasing* artifacts (image jaggedness, due for instance to the robot moving quickly), ensuring smoother rendering.

5 3D Semantic Reconstruction

5.1 Overview

3D Semantic Reconstruction/Labeling is a technique used in robotics to reconstruct and label 3D reconstructions, assigning a certain color to distinguish objects based on their semantic properties. This adds a critical layer of informative context for autonomous operations. Modern methods extend this technique using **Gaussian Splatting**, which represents each point in a point cloud with a Gaussian kernel to create a continuous and detailed surface.

5.2 3D Semantic Reconstruction Processes

5.2.1 3D Semantic Labeling

2D semantic labeling categorizes perceived objects in a video and produces a color to indicate the object type. This 2D labeling (often trained on datasets like KITTI and Semantic3D) is applied to scene reconstructions to show the dimensions of certain objects within the reconstruction.

3D semantic reconstructions come in two forms:

- **Sparse Reconstructions**: Typically consist of a sparse point cloud, where frame-by-frame 2D labeled scenes are augmented to assign each point a color corresponding to the object it represents. These provide sufficient information for robotic systems to understand and navigate.
- **Dense Reconstructions**: Use methods like *marching cubes* algorithms and dense point maps to produce detailed meshes. For example, **voxelization** methods are used to reconstruct the scene and project colors from the semantic map onto the reconstruction. The truncated signed distance function (**TSDF**) is used to highlight surfaces with a specific color, enabling a robot to navigate and clean selected surfaces.

5.2.2 Applications in Environments

- **Indoor**: Focuses on labeling objects such as chairs and tables for robot recognition and interaction, although processing and labeling thin objects (e.g., chair legs) can be problematic.
- **Outdoor and Collaborative**: The use of multiple robots (for example, a car and a drone) can reduce the time required for **sparse reconstruction**. Robots can scan an area and semantically fuse all scans to produce a global map. However, some multi-robot methods produce **incomplete sparse maps**, which are unsuitable for high-quality dense reconstructions.

6 Multi-View 3D Reconstruction

6.1 Overview

The use of **multiple views** of an object generally improves the overall quality of the reconstruction. Many approaches use a trajectory to produce multiple views of the perceived object, often employing **Structure from Motion (SfM)** algorithms. Other methods reconstruct objects from images taken from different positions while the acquisition device is in a fixed position or moved on a robotic arm.

6.2 Reconstruction and Fusion Techniques

- **Viewpoints and Sensors**: Scenes are captured using **stereo** and **monocular** cameras. Point clouds are favored for a lower cost factor in performance on computerized devices.
- **Triangulation and Mapping**: Camera frames can be correlated and fused via triangulation or by developing a depth map to create a mesh, using GPS data to fuse reconstructions.
- **Fusion Techniques**:
  - **Structure from Motion (SfM)**: Takes multiple views without known mutual positions and matches them to produce a reconstruction. This method is used by monocular cameras because it does not rely on reliable distances between lenses.

  - **TSDF Fusion (Truncated Signed Distance Function)**: Uses multiple depth maps from views perceived by stereo cameras to produce high-quality reconstructions.
  - **Zero-Shot Amodality**: **DUSt3R** eliminates the need for known camera parameters or poses by directly regressing (through a neural net) 3D point maps from pairs of images, enabling dense reconstruction without prior camera information.
  - **Detector-Free SfM**: Addresses the limitations of *keypoint*-based SfM (difficult in texture-poor scenes) by eliminating the need for explicit *keypoint* detection and using *detector-free matchers* to establish correspondences (the network learns to find geometric correspondences between two images even in visually uniform areas).

7 Amodal 3D Reconstruction

7.1 Overview

Amodal reconstruction takes the perceived view and reconstructs it, then uses a **Neural Network** to **complete the object** in relation to a database of objects. The primary goal of this method is to improve **object manipulation** by robots.

- **Representation**: Some methods use **point clouds** (computationally less expensive but reduce quality), while others use TSDF mesh fusion (computationally heavy but accurate).
- **Physical Realism**: **ARM** uses *priors* (prior knowledge: e.g., it recognizes the shape of a table) on physical properties (connectivity and objects: it knows that solid objects are continuous and not disconnected points) to increase the physical realism of amodal reconstructions.

7.2 Amodal Reconstruction Processes

7.2.1 Reconstruction Methods

Sparse and dense point cloud methods, fused TSDF models, and voxel maps are used. Two reconstructions occur: one for the perceived object (incomplete shape) and one for shape completion (reconstructs the incomplete part).

7.2.2 Shape Completion

Shape completion estimates the missing parts of an incomplete reconstruction to improve object manipulation. It typically employs a neural network, trained on an object database (e.g., YCB or ShapeNet).

- **Grasp-Oriented**: The network can focus on predicting grasp positions. For example, one method uses a perceived dense point cloud and a sparse estimation of missing parts to predict grasp positions (it does not reconstruct the object but directly predicts the movements to grab it).
- **Reconstruction-Oriented**: The network can improve object reconstruction for better manipulation. For example, voxelized shape completion of objects uses a perceived TSDF model (it does not predict movements but reconstructs the object in a way that is useful for the robot).
- **Accuracy**: Accuracy can be improved using custom reference databases, such as an algorithm that uses a custom reference database of reconstructed fruits and vegetables (CoRe) to improve the shape of completed objects.

8 Motion Planning and Collision

NeRFs are used to guarantee collision-free paths for robots.

- **Vision-Only Navigation**: Paths can be flawlessly tracked and executed without collisions using only RGB camera input.
- **Uncertainty Quantification**: A transformative approach models NeRFs into equivalent **Poisson Point Processes (PPPs)** to rigorously quantify uncertainty and compute collision probabilities. This uses a voxel representation called **Probabilistic Unsafe Robot Region (PURR)** to facilitate rapid trajectory optimization.

8.1 Video Compression and Motion Estimation

NeRFs have potential for video compression and motion estimation in industrial and robotic fields.

- **Motion Estimation**: The **Dynamic-NeRF (D-NeRF)** model can accurately estimate depth information and motion dynamics (for example, of robotic arms).
- **Video Compression**: NeRF-based techniques can achieve compression savings of up to 48% for high-resolution video, facilitating faster data transfer and improving real-time performance in robotics.

9 Datasets

Datasets are classified based on application areas (e.g., human interaction, objects/shapes, scenes, and semantics).

- **Semantic Reconstruction**: Uses datasets like **KITTI** and **ADE20K**, known for their wide range of semantically labeled scenes and objects.
- **Amodal Reconstruction/Grasping**: Employs datasets like **YCB** and **ShapeNet** to cover a wide range of objects, facilitating complete object reconstruction for accurate grasping.
- **SLAM and Multi-View**: Datasets like **EuRoC** and **TUM-RGBD** are widely used for SLAM and visual odometry research, providing accurate ground truth.
- **NeRF**: The **NeRF Synthetic** dataset is specifically designed to evaluate NeRF models, featuring synthetic 3D scenes and objects rendered from different viewpoints.

10 Experimental Results and Discussion

10.1 SLAM Results

SLAM has demonstrated a **significantly reduced runtime** compared to other reconstruction methods. The use of **edge computing** techniques is critical to further reducing the execution time of the SLAM algorithm.

- **Sensor Accuracy**: Stereo and RGB-D systems outperform monocular approaches in metrics like TRMSE and ATE.
- **Collaborative Efficiency**: RecSLAM and Collaborative SLAM utilize traditional SLAM mapping techniques and sparse reconstruction.

10.2 Gaussian Splatting Results

Gaussian-based methods show high performance.

- **Reconstruction Quality**: In the Replica environment, HF-SLAM outperforms other methods in reconstruction with a PSNR of 35.74.
- **Grasp Success Rate**: ManiGaussian achieves a grasp success rate of 44.8%, while Splat-MOVER excels with a success rate of 100%.

10.3 3D Semantic Reconstruction Results

Dense reconstruction methods (e.g., RTSDM) achieve a high IoU (85.1%) with a low ATE (0.11345 m). Sparse methods (e.g., Kimera-Multi) are generally faster but can vary in accuracy.

10.4 Amodal Reconstruction Results

The **Point Cloud Shape Completed** approach (kPAM-SC) shows a significant improvement in grasp success rate (96%), compared to Voxel/Mesh methods (e.g., ARM with 42%).

10.5 NeRF Results

Dex-NeRF achieves the highest manipulation success rate (96.6%). For navigation, **CollisionNeRF** (83.0%) and **NeRF2Real** (78.0%) demonstrate high success rates in navigation tasks.

11 Conclusions and Future Challenges

Developments like SLAM and NeRF have significantly boosted robotic capabilities in understanding and interacting with complex environments. However, significant hurdles persist:

- **Computational Intensity**: Computational complexity and data integration often hinder real-time processing.
- **Limited Datasets**: The lack of diverse and comprehensive datasets representing the variety of scenarios robots might encounter is a significant limitation.
- **Occlusion Management**: Current methods struggle to accurately handle **occlusions** and **partial views**.
- **Sensor Integration**: Lack of integration between 3D reconstruction algorithms and advanced sensor technologies, such as thermal imaging.

Future research should focus on integrating AI to improve semantic understanding and developing algorithms that optimize computational efficiency. Exploring unsupervised learning methods could also enable robots to adapt more efficiently to new environments.

12 Terminology

12.1 Gaussian Splatting (GS): Explicit Representation

Gaussian Splatting (GS) is a 3D reconstruction and rendering technique that explicitly models a scene as a collection of **millions of elementary primitives**: **3D Gaussian kernels**. This explicit approach offers notable speed advantages over implicit methods like NeRF.

1. The 3D Gaussian Kernel

In GS, a single Gaussian is not a statistical curve, but a **blurry, ellipsoidal 3D shape** whose parameters are optimized to represent a tiny fragment of matter in space. Each Gaussian i is defined by a set of explicit parameters:

- **Central Position (μ_i):** The (x, y, z) coordinates of the center.
- **Covariance (Σ_i):** A matrix that defines the **shape** (scale on the three axes) and the **orientation** of the 3D kernel.
- **Color and Opacity:** Color ($\mathbf{c}_i$) and opacity (α_i).

2. Training as Differentiable Optimization

Training does not optimize the weights of an MLP network (like NeRF), but optimizes the **explicit parameters of each individual Gaussian**.

1. **Initialization:** Training begins with a **sparse 3D point cloud** (generated by SfM), where each point provides the initial position (μ) for a Gaussian.
2. **Optimization:** The process uses **Backpropagation** to compute the gradient of the *Loss Function* (color error) with respect to all parameters of all Gaussians. This allows the system to "fit" and sculpt the shape and position of each ellipsoid until their combination perfectly reproduces the 2D training images.

3. Rendering Efficiency (Splatting)

The speed of GS stems from its rendering method, **Splatting**, which is much more efficient than NeRF's ray-based *Volume Rendering*:

- **3D → 2D Conversion:** 3D Gaussians are directly **projected** onto the screen as 2D Gaussian kernels (ellipses), attempting to fit these ellipses to the cloud points.
- **Accelerated Rasterization:** These 2D ellipses are sorted by depth (*Z-sorting*) and accumulated (blended) to produce the final pixel color. Because it relies primarily on geometric processing and accumulation, the process leverages **GPU Hardware acceleration** (similar to traditional graphics rasterization), reducing rendering times from seconds to **milliseconds** (real-time performance).

In summary, GS sacrifices the continuous implicit representation of NeRF in favor of an explicit, manipulable representation that allows extremely rapid rendering.

12.2 Neural Radiance Fields (NeRF)

Neural Radiance Fields (NeRF) are a *novel view synthesis* technique that uses a neural network to represent a 3D scene implicitly and continuously.

1. The 5D Implicit Representation

Unlike explicit representations (such as meshes or point clouds), NeRF models the scene as a continuous function $\mathcal{F}$ encoded in the weights of a ***Fully Connected*** **neural network (MLP)**.

The input of the function is a 5-dimensional vector, and the output is a pair of values:

$$\mathcal{F} : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$$

Where:

- $\mathbf{x} = (x, y, z) \in \mathbb{R}^3$: **Position** 3D in space.
- $\mathbf{d} = (\theta, \phi) \in \mathbb{R}^2$: **Viewing Direction**.
- $\mathbf{c} \in \mathbb{R}^3$: RGB **Color** emitted at $\mathbf{x}$ as seen from direction $\mathbf{d}$.
- $\sigma \in \mathbb{R}$: **Volumetric Density** (or differential opacity) at $\mathbf{x}$.

To capture high-frequency details (textures), the coordinates $\mathbf{x}$ and $\mathbf{d}$ are first mapped into a high-dimensional space via **Positional Encoding**.

2. Volume Rendering

The synthesis of a view from a virtual camera is achieved via **Volume Rendering**. For each pixel of an image to be rendered, a light ray r is cast through the scene, and between 100-200 points are sampled along the ray. The final color $\mathbf{C}(r)$ of the pixel is calculated by integrating the color and density values of the points predicted by the network along the ray:

$$\mathbf{C}(r) = \int_{t_n}^{t_f} T(t) \cdot \sigma(\mathbf{r}(t)) \cdot \mathbf{c}(\mathbf{r}(t), \mathbf{d})\, dt$$

Where $T(t)$ is the **Transmittance** function (the probability that light reaches point t without being occluded) and $\mathbf{r}(t)$ is the 3D point on the ray.

3. Training and Loss Function

NeRF is trained on a set of 2D images with known camera poses. Training is a process of **overfitting** to the specific scene.

The network is optimized to minimize the **Loss Function** ($\mathcal{L}$), typically the Mean Squared Error (L_2 Loss), between the predicted (rendered) color and the true color of the pixel:

$$\mathcal{L} = \sum_{r} \|\mathbf{C}_{predicted}(r) - \mathbf{C}_{true}(r)\|^2$$

The error is backpropagated through the Volume Rendering process to update the weights of the network via **Gradient Descent**, forcing the network to encode the correct geometry and illumination of the scene.

4. Reconstruction Constraint

A crucial aspect of NeRF is that it is a **scene-specific** model. If the scene changes (e.g., from an office to a garden), it is necessary to **completely retrain the network** on the new data.

12.3 Truncated Signed Distance Function (TSDF)

The **TSDF (Truncated Signed Distance Function)** is a volumetric geometric representation widely used in real-time SLAM (e.g., KinectFusion) to reconstruct dense 3D scenes from depth sensors.

1. Volumetric Representation

TSDF models the scene as a 3D grid of discrete cubes called **voxels**. Each voxel v stores two main values:

- **TSDF Value:** The distance from the voxel to the nearest surface.
- **Weight (W):** The reliability or number of measurements that contributed to this value.

2. Signed Distance Function (SDF)

The TSDF value encodes the voxel's relation to the surface:

- **Positive (> 0):** The voxel is located in **free space** (between the sensor and the measured surface).
- **Negative (< 0):** The voxel is located in **occupied space** (the inside of the object, assuming solidity).
- **Zero ($= 0$):** The voxel is located **on the surface**.

The **Truncation (τ)** limits calculation and storage only to voxels that lie within a small distance band ($\pm\tau$) from the surface, improving efficiency.

3. Surface Identification via Fusion

The surface is not identified by a single noisy measurement, but by the **statistical convergence** of multiple measurements:

- **Weighted Fusion:** When a new depth ray is acquired, it updates a column of voxels around the impact point. The new TSDF value is calculated using a **weighted average** between the old value and the new estimate, which dampens sensor noise.

$$D_{fusion} = \frac{W_{old} \cdot D_{old} + W_{new} \cdot D_{new}}{W_{old} + W_{new}}$$

- **Zero-Crossing:** The final 3D surface (extracted via *Marching Cubes*) is defined by the geometric locus of all voxels that have a **TSDF value equal to zero**. This 0 represents the most reliable statistical position between positive space (outside) and negative space (inside).

12.4 Structure from Motion (SfM)

Structure from Motion (SfM) is a computer vision technique that reconstructs the 3D geometry of a scene (*Structure*) and estimates the position and orientation of all cameras (*Motion*) from an unordered set of 2D images. It is the essential precursor for dense reconstruction (such as NeRF or GSplat).

1. Phases of the SfM Process

The process relies on finding geometric correspondences and optimization:

1. **Feature Detection and Matching:** Distinctive **keypoints** are identified in each image (using algorithms like SIFT or ORB) and **correspondences** are established between different views.
2. **Triangulation:** Using the 2D correspondences and the estimated pose of the cameras, the 3D position of the keypoints in space is computed (see below).
3. **Initial Pose Estimation:** The **Essential and Fundamental Matrices** problem is solved to estimate the relative position and orientation of the cameras. This provides the crucial input for triangulation.
4. **Bundle Adjustment (Global Optimization):** This is the final refinement stage. It is an optimization technique that minimizes **reprojection** error (the difference between where a 3D point projects onto the image and where it was actually detected as a *keypoint*). BA simultaneously optimizes all camera poses and all 3D point positions to achieve maximum geometric consistency.

2. The Mechanism of Triangulation

Triangulation is the core of 3D computation. It does not require ray length as an input but computes it as an output.

- **Input:** The known **3D pose** of at least two cameras (their position and orientation $R|t$) and the 2D coordinates of a corresponding point on both images.
- **Principle:** For each 2D point on an image, a **projective ray** originating from the camera's optical center can be defined.
- **Output:** The real 3D position of the point in the scene (P) is given by the **intersection** of two or more projective rays. Since in the real world rays do not intersect perfectly due to noise, triangulation computes the 3D coordinate $P = (X, Y, Z)$ that **minimizes the distance** between the two rays in space, thus resolving the uncertainty regarding **depth** (ray length).

13 References

References

[1] Selvaratnam, D., Bazazian, D. (2025).
3D Reconstruction in Robotics: A Comprehensive Review.
Computers & Graphics, 130, 104256.
https://doi.org/10.1016/j.cag.2025.104256